

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Компьютерная графика»

**ТЕМА: «Исследование алгоритмов отсечения отрезков и
многоугольников окнами различного вида»**

Студенты гр. 3311

Аршин А.Д.
Баймухамедов
Р.Р.
Пасечный Л.В.

Преподаватель

Колев Г. Ю.

Санкт-Петербург

2025

Цель работы:

Задание (вариант 3):

Обеспечить реализацию алгоритма отсечения массива произвольных отрезков заданным прямоугольным окном с использованием алгоритма КознаСазерленда. Вначале следует вывести на экран сгенерированные отрезки полностью, а затем другим цветом или яркостью те, которые полностью или частично попадают в область окна.

Теоретическая основа:

Самой простой задачей отсечения является задача отсечения отрезков. Сформулируем её на конкретном примере. Будем считать, что область вывода задаётся прямоугольником ABCD. Рассмотрим пример, и в качестве отсекаемой фигуры возьмём треугольник PRQ.

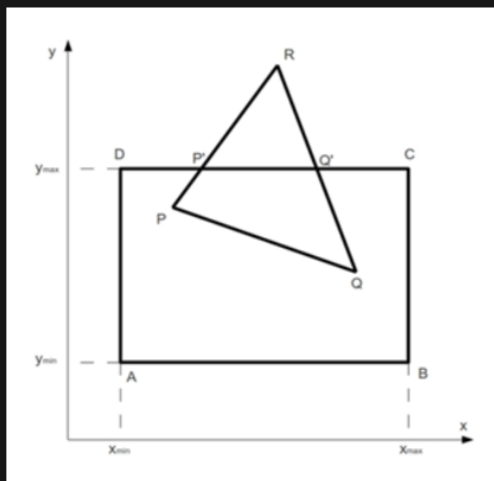


Рис. 1

Отсечение – это процесс, связанный с выделением и визуализацией фрагмента плоской или пространственной сцены, расположенного внутри (*внутреннее отсечение*) или, наоборот, вне (*внешнее отсечение*) некоторой соответственно двумерной или трехмерной отсекающей фигуры (*отсекателя*). Оставшаяся часть сцены при этом игнорируется, т.е. визуализации не подлежит. При реализации такого процесса используются алгоритмы *двумерного* или *трехмерного отсечения*. Специфика конкретного алгоритма определяется, в первую очередь, формой отсекающей фигуры, которая может быть регулярной (стандартной, такой, например, какую имеют прямоугольник или прямоугольный параллелепипед) или нерегулярной (нестандартной). При регулярной форме отсекаателя важным фактором является также его ориентация относительно системы координат.

Как уже отмечалось, изображение в компьютерной графике формируется с использованием, прежде всего, отрезков. Совокупностями определенным образом ориентированных отрезков представляются не только ломаные линии, многоугольники и многогранники, но также разомкнутые и замкнутые кривые. Трехмерные поверхности аппроксимируют наборами плоских полигонов, ограниченных многоугольниками (чаще всего, треугольниками), т.е. опять же наборами отрезков. Поэтому и алгоритмы отсечения работают, в первую очередь, с отрезками.

Алгоритм двумерного отсечения Сазерленда-Козна

Данный алгоритм реализует отсечение отрезков координатно-ориентированным прямоугольником, границы которого: левая, правая, нижняя и верхняя – задаются координатами соответственно x_L , x_H , y_H и y_L (рис.8.1).

При таком отсечении отсекатель часто называют **отсекающим окном**. Рассмотрим сначала основные принципы построения алгоритма применительно к внутреннему отсечению. Точка $P(x, y)$ находится внутри отсекающего окна, если ее координаты удовлетворяют двум условиям:

$$x_L \leq x \leq x_H; \\ y_L \leq y \leq y_H$$

(здесь и далее принадлежность точки границе отсекателя приравнивается случаю ее расположения внутри него).

Если для обоих концов отрезка эти условия выполняются, он целиком расположен внутри отсекающего окна, т.е. является, как говорят, полностью видимым (как отрезок ab на рис.8.1). Если оба конца отрезка находятся левее отсекающего окна (их горизонтальные координаты удовлетворяют неравенству $x < x_L$, как у отрезка cd) отрезок безусловно невидим. То же самое можно сказать про отрезки, расположенные целиком правее, ниже или выше отсекателя (для обоих концов отрезков при этом справедливы неравенства соответственно $x > x_H$, $y < y_L$ или $y > y_H$). Сложности возникают при отсечении таких отрезков, как ef и gh . Изначально они не идентифицируются как полностью видимые или безусловно невидимые. В действительности же каждый из таких отрезков может быть частично видимым (как отрезок ef) или полностью невидимым (как отрезок gh). Поэтому при анализе подобных отрезков необходимы какие-либо формальные методы, позволяющие определить реальные точки пересечения отрезков с границами отсекающего окна или установить, что они отсутствуют. Задача нахождения реальной точки пересечения отрезка с какой-либо одной границей отсекателя возникает и в случае, когда один из концов отрезка расположен внутри отсекающего окна, а второй – вне его (как у отрезка gh).

В алгоритме Сазерленда-Козна при обработке отрезков используются четырехрядные (битовые) коды, определяющие расположение концов отрезков относительно границ отсекающего окна – концевые коды. При формировании кода конца отрезка с координатами (x, y) в первый (крайний правый) бит заносится 1, если $x < x_L$, во второй – если $x > x_H$, в третий – если $y < y_L$, в четвертый – если $y > y_H$. В остальных случаях в соответствующие биты заносится 0. Таким образом, вся координатная плоскость разбивается на девять областей, в каждой из которых концы отрезка присваивается уникальный концевой код (см. рис.8.1). Выявить целиком видимый отрезок при этом достаточно просто: оба концевых кода у него полностью нулевые (как у отрезка ab на рис.8.1). Если отрезок не идентифицирован как целиком видимый, для него определяется также побитовое логическое произведение концевых кодов. Очевидно, что у отрезков, целиком расположенных левее, правее, ниже или выше отсекающего окна, в какой-либо один разряд обоих концевых кодов при их формировании будет занесено по 1; это обусловит наличие 1 в соответствующем разряде побитового логического произведения этих концевых кодов (так, у отрезка cd , расположенного целиком левее отсекающего окна, с кодами концов 0101 и 0001 побитовое логическое произведение равно 0001). Поэтому отличное от нуля побитовое логическое произведение является признаком безусловной невидимости отрезка. Вместе с тем, при полностью нулевом побитовом произведении концевых кодов отрезок может оказаться частично видимым (как отрезки ef и gh) или полностью невидимым (как отрезок ij). Для подобных отрезков при выявлении их видимых частей или идентификации как полностью невидимых используются результаты расчета точек пересечения бесконечной прямой линии, проведенной через отрезок, с бесконечными прямыми линиями, проведенными через ребра отсекающего окна.

Рис. 2

Для отрезка с началом в точке $P_1(x_1, y_1)$ и концом в точке $P_2(x_2, y_2)$, если он не вертикален ($m = (y_2 - y_1)/(x_2 - x_1) \neq \infty$), такие точки пересечения на прямых, проведенных через левое и правое ребра, будут иметь координаты соответственно:

$$x = x_1, \quad y = m(x_2 - x_1) + y_1; \quad x = x_n, \quad y = m(x_n - x_1) + y_1.$$

Если отрезок не горизонтален ($m \neq 0$), координаты аналогичных точек пересечения на прямых, проведенных через нижнее и верхнее ребра, будут равны соответственно:

$$x = (y_n - y_1)/m + x_1, \quad y = y_n; \quad x = (y_n - y_1)/m + x_1, \quad y = y_n.$$

Последовательность действий в алгоритме двумерного внутреннего отсекаания Сазерленда-Козна следующая. С использованием концевых кодов отрезок проверяется на полную видимость и безусловную невидимость. Если проверка не дает очевидного результата, поочередно работают четыре бесконечные отсекающие прямые, проходящие соответственно через левое, правое, нижнее и верхнее ребра отсекающего окна. В каждом случае проверяется, не лежит ли отрезок полностью в той же стороне от отсекающей прямой, что и само окно (очевидно, например, что если в первом бите кодов концов отрезка содержится θ , отрезок полностью лежит правее левой отсекающей прямой). Если это так, переходят к следующей отсекающей прямой. В противном случае проверяют, не лежит ли начало отрезка в той же стороне, что и окно. Если это подтверждается, начало и конец отрезка меняются местами. Начало отрезка в любом случае оказывается по отношению к отсекающей прямой в стороне, противоположной той, в которой находится окно. После определения точки пересечения прямой, проведенной через отрезок, с текущей отсекающей прямой начало отрезка переносится в эту точку (удаляется часть отрезка). Полученный новый отрезок вновь проверяется на полную видимость и безусловную невидимость. В зависимости от результата переходят к следующей отсекающей прямой или алгоритм заканчивает работу.

Практическая реализация алгоритма двумерного отсекаания Сазерленда-Козна представлена на следующих примерах

[Пример 1](#)

[Пример 2](#)

Алгоритм трехмерного отсекаания Сазерленда-Козна

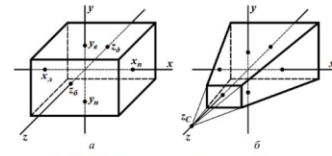
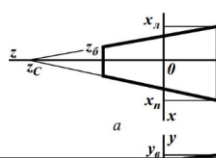


Рис.8.3. Объемные отсекатели регулярной формы: прямоугольный параллелепипед (а) и пирамида видимости (б)

При помощи этого алгоритма можно осуществлять отсекаание отрезков двумя наиболее распространенными объемными отсекателями регулярной формы: координатно-ориентированным прямоугольным параллелепипедом (рис.8.2а) и усеченной пирамидой (рис.8.2б), называемой также пирамидой видимости. Каждый из названных отсекателей имеет шесть граней: левую, правую, нижнюю, верхнюю, ближнюю и дальнюю. Так же, как и при двумерном отсекании, положение каждого конца отсекаемого отрезка относительно границ отсекателя (точнее – относительно бесконечных плоскостей, проходящих через его грани) можно связать с соответствующим концевым кодом, имеющим в данном случае шесть разрядов (бит).

Границы первого отсекателя – прямоугольного параллелепипеда – задаются шестью координатами: x_1, x_n, y_n, y_n, z_n и z_n (рис.8.2а). При этом процесс формирования кода конца отрезка с координатами (x, y, z) полностью аналогичен тому, который применяется при двумерном отсекании: I заносится в первый (крайний правый) бит, если $x < x_1$ (конец левее отсекателя), во второй – если $x > x_n$ (конец правее отсекателя), в третий – если $y < y_n$ (конец ниже отсекателя), в четвертый – если $y > y_n$ (конец выше отсекателя), в пятый – если $z > z_n$ (конец ближе отсекателя), в шестой – если $z < z_n$ (конец дальше отсекателя); в остальных случаях в соответствующие биты заносится θ .



При использовании в качестве отсекателя усеченной пирамиды концевые коды формируются с учетом специфики геометрии данного объемного тела. Допустим, что она задана следующим образом (рис.8.4а и 8.4б): z_c – z -координата центра проекции, расположенного на оси z ; z_n и z_n – координаты соответственно ближней и дальней граней по оси z ; x_1 и x_n – координаты соответственно левой и правой граней по оси x при $z = z_n$ (см. рис.8.4а); y_n и y_n – координаты соответственно нижней и верхней граней по оси y при $z = z_n$ (см. рис.8.4б).

Для анализа положения конца отрезка относительно левой, правой, нижней и верхней граней отсекающей пирамиды используются так называемые пробные функции. Идея их составления довольно проста. Уравнения плоскостей, несущих указанные грани, будут выглядеть соответственно так (см. рис.8.4):

Рис.3

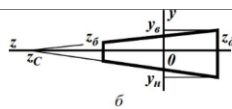


Рис.6.4. Пирамида видимости: виды сверху (а) и справа (б)

Пробные функции применительно к этим граням совпадают с левыми частями приведенных уравнений:

$$x - x_1 \frac{z_c - z}{z_c - z_n} = \theta; \quad x - x_n \frac{z_c - z}{z_c - z_n} = \theta;$$

$$y - y_n \frac{z_c - z}{z_c - z_n} = \theta; \quad y - y_n \frac{z_c - z}{z_c - z_n} = \theta.$$

$$f_3 = x - x_1 \frac{z_c - z}{z_c - z_n}; \quad f_4 = x - x_n \frac{z_c - z}{z_c - z_n};$$

$$f_5 = y - y_n \frac{z_c - z}{z_c - z_n}; \quad f_6 = y - y_n \frac{z_c - z}{z_c - z_n}.$$

Значение пробной функции после подстановки в нее координат конца отрезка (x, y, z) позволяет определить положение этого конца относительно соответствующей грани и, в свою очередь, заполнить надлежащим образом соответствующий бит концевой кода. При внутреннем отсекании это осуществляется так.

Если $f_3 < \theta$, конец расположен левее плоскости левой грани, и в первый бит концевой кода заносится I ; если же $f_3 = \theta$ или $f_3 > \theta$, конец расположен соответственно на плоскости левой грани или правее нее, и в первый бит заносится θ .

Если $f_4 > \theta$, конец расположен правее плоскости правой грани, и во второй бит концевой кода заносится I ; если же $f_4 = \theta$ или $f_4 < \theta$, конец расположен соответственно на плоскости правой грани или левее нее, и во второй бит заносится θ .

Если $f_5 < \theta$, конец расположен ниже плоскости нижней грани, и в третий бит концевой кода заносится I ; если же $f_5 = \theta$ или $f_5 > \theta$, конец расположен соответственно на плоскости нижней грани или выше нее, и в третий бит заносится θ .

Если $f_6 > \theta$, конец расположен выше плоскости верхней грани, и в четвертый бит концевой кода заносится I ; если же $f_6 = \theta$ или $f_6 < \theta$, конец расположен соответственно на плоскости верхней грани или ниже нее, и в четвертый бит заносится θ .

Аналогичный подход применим и для заполнения пятого и шестого битов концевых кодов с учетом положения концов отрезка относительно ближней и дальней граней. Уравнения плоскостей, проходящих через эти грани, выглядят соответственно так:

$$z - z_n = \theta; \quad z - z_n = \theta.$$

$$f_7 = z - z_n; \quad f_8 = z - z_n.$$

Вид соответствующих пробных функций очевиден:

Если после подстановки в первую из них z -координаты конца отрезка $f_7 > \theta$, конец расположен перед плоскостью ближней грани, и в пятый бит концевой кода заносится I ; если же $f_7 = \theta$ или $f_7 < \theta$ конец расположен соответственно на плоскости ближней грани или за ней, и в пятый бит заносится θ .

Если после такой же подстановки во вторую пробную функцию оказывается, что $f_8 < \theta$, конец расположен за плоскостью дальней грани, и в шестой бит концевой кода заносится I ; если же $f_8 = \theta$ или $f_8 > \theta$ конец расположен соответственно на плоскости дальней грани или перед ней, и в шестой бит заносится θ .

Таким образом, если отсекаание осуществляется прямоугольным параллелепипедом, то все координатное пространство разбивается на двадцать семь областей, в каждой из которых концу отрезка присваивается уникальный код.

Если отсекателем является усеченная пирамида, координатное пространство разбивается в общем случае на тридцать шесть областей с уникальным кодом конца отрезка в каждой из них: двадцать семь областей – спереди от центра проекции, т.е. перед точкой наблюдения, и девять – позади от центра проекции, т.е. за точкой наблюдения.

Вместе с тем, представляется целесообразным использовать иной подход к решению этой задачи. Все, что находится позади от центра проекции, наблюдатель видеть не может. Поэтому, независимо от того, какое именно отсекаание реализуется – внутреннее или внешнее, начать обработку трехмерной сцены можно с удаления ее части, расположенной за центром проекции, т.е., иными словами, в полупространстве $z > z_c$ (здесь и

Рис.4

Выполнение лабораторной работы

В качестве языка программирования для выполнения данной лабораторной работы выберем Python. Для быстрого запуска достаточно иметь установленный Python 3.

Демонстрация (<https://disk.yandex.ru/i/7AfHlUoG7CDdwQ>)

Интерфейс созданного приложения:

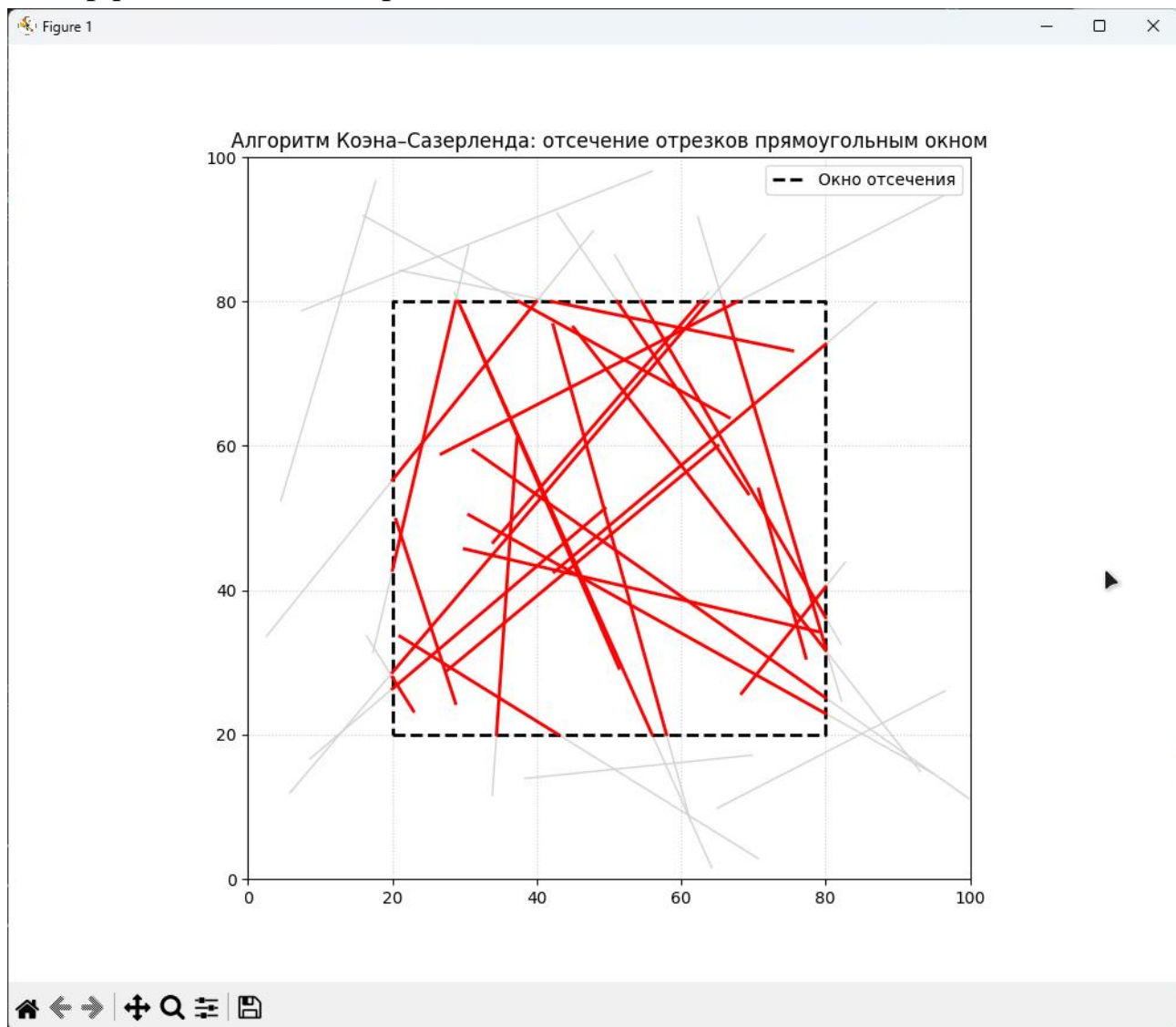


Рис. 3 Готовое приложение 1

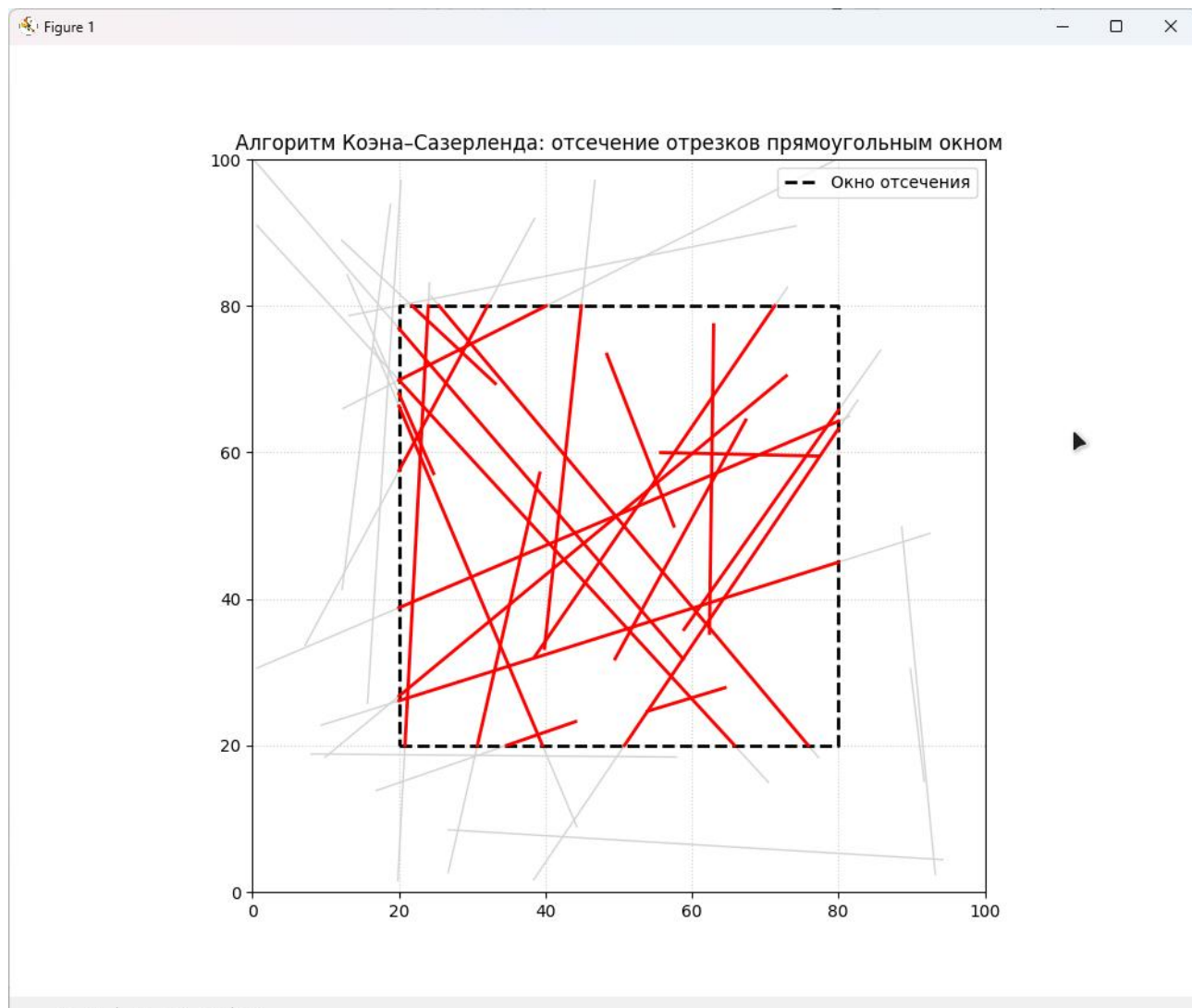


Рис. 4 Готовое приложение 2

Как мы видим исходя из рисунков, при каждом запуске программы у нас создаются случайно прямые линии, которые отсекаются фиксированным окном

Вывод:

В ходе выполнения работы был успешно реализован и протестирован алгоритм Козна-Сазерленда для отсечения отрезков прямоугольным окном. Анализ результатов позволяет сделать следующие выводы:

1. **Эффективность алгоритма:** Алгоритм Козна-Сазерленда доказал свою эффективность для быстрого отбрасывания тривиальных случаев - отрезков, полностью невидимых или полностью видимых, что значительно сокращает количество вычислений пересечений.
2. **Корректность работы:** Алгоритм корректно обрабатывает все возможные случаи расположения отрезков:
 - Полностью видимые отрезки (отображаются целиком)
 - Полностью невидимые отрезки (не отображаются)
 - Частично видимые отрезки (отображаются только видимые части)
3. **Визуальная демонстрация:** Реализованная визуализация наглядно

показывает принцип работы алгоритма - все исходные отрезки отображены серым цветом, а их видимые части выделены красным, что позволяет легко оценить корректность отсечения.

4. **Стабильность алгоритма:** Алгоритм устойчиво работает с отрезками различной ориентации и длины, включая вырожденные случаи и отрезки, параллельные границам окна.
5. **Практическая ценность:** Освоенный алгоритм является фундаментальным в компьютерной графике и может быть применен в различных задачах, требующих отсечения геометрических примитивов, таких как визуализация 3D-сцен, обработка географических данных и разработка графических интерфейсов.

Приложение 1: Листинг программы

```
import matplotlib.pyplot as plt
import random

# Определяем битовые маски для четырёх границ прямоугольного окна
#TOP – точка выше верхней границы (y > ymax)
#BOTTOM – ниже нижней (y < ymin)
#RIGHT – правее правой (x > xmax)
#LEFT – левее левой (x < xmin)
TOP = 8    # 1000
BOTTOM = 4 # 0100
RIGHT = 2  # 0010
LEFT = 1   # 0001

#Функция вычисления кода точки. принимает координаты точки (x, y) и границы окна
def compute_code(x, y, xmin, xmax, ymin, ymax):
    # Вычисляет 4-битовый код точки относительно окна.
    code = 0
    # если точка левее окна
    if x < xmin:
        code |= LEFT
    # Если точка правее окна
    elif x > xmax:
        code |= RIGHT
    # если точка ниже окна
    if y < ymin:
        code |= BOTTOM
    # если точка выше окна
    elif y > ymax:
        code |= TOP
    # возвращаем итоговый 4-битовый код точки
    return code

# функция алгоритма Коэна-Сазерленда
def cohen_sutherland_clip(x1, y1, x2, y2, xmin, xmax, ymin, ymax):
    # Вычисляем коды для обоих концов отрезка
    code1 = compute_code(x1, y1, xmin, xmax, ymin, ymax)
    code2 = compute_code(x2, y2, xmin, xmax, ymin, ymax)
    # Флаг для проверки попадания отрезка в окно полностью или частично
```

```

accept = False

# цикл пока не примем или отбросим отрезок
while True:
    # условие побитого ИЛИ равно 0
    if not (code1 | code2): # оба конца внутри окна
        accept = True
        break
    # Если побитовое И не равно 0
    elif code1 & code2: # оба конца в одной внешней области – отбрасываем
        break
    # Иначе отрезок будет частично виден
    else:

        # Выбираем внешнюю точку для пересчёта
        x, y = 0, 0
        code_out = code1 if code1 != 0 else code2

        # Находим точку пересечения с границей окна
        # Используем параметрическое уравнение прямой
        if code_out & TOP:
            x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1)
            y = ymax
        elif code_out & BOTTOM:
            x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1)
            y = ymin
        elif code_out & RIGHT:
            y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1)
            x = xmax
        elif code_out & LEFT:
            y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1)
            x = xmin

        # Заменяем внешнюю точку на точку пересечения
        # Пересчитываем ее код. Отрезок укорочен
        if code_out == code1:
            x1, y1 = x, y
            code1 = compute_code(x1, y1, xmin, xmax, ymin, ymax)
        else:
            x2, y2 = x, y
            code2 = compute_code(x2, y2, xmin, xmax, ymin, ymax)

    if accept:
        return (x1, y1, x2, y2)
    else:
        return None

# Параметры окна отсечения
xmin, xmax = 20, 80
ymin, ymax = 20, 80

# Генерация случайных отрезков
num_segments = 30
segments = []

```

```
for _ in range(num_segments):
    x1 = random.uniform(0, 100)
    y1 = random.uniform(0, 100)
    x2 = random.uniform(0, 100)
    y2 = random.uniform(0, 100)
    segments.append((x1, y1, x2, y2))

# Визуализация
plt.figure(figsize=(10, 8))

# Рисуем все исходные отрезки серым цветом
for (x1, y1, x2, y2) in segments:
    plt.plot([x1, x2], [y1, y2], color='lightgray', linewidth=1)

# Рисуем окно отсечения
plt.plot([xmin, xmax, xmax, xmin, xmin],
        [ymin, ymin, ymax, ymax, ymin],
        color='black', linewidth=2, linestyle='--', label='Окно отсечения')

# Применяем алгоритм Коэна-Сазерленда и рисуем видимые части отрезков
# Если результат не None – рисуем видимую часть красным.
for (x1, y1, x2, y2) in segments:
    clipped = cohen_sutherland_clip(x1, y1, x2, y2, xmin, xmax, ymin, ymax)
    if clipped:
        xc1, yc1, xc2, yc2 = clipped
        plt.plot([xc1, xc2], [yc1, yc2], color='red', linewidth=2)

plt.xlim(0, 100)
plt.ylim(0, 100)
plt.gca().set_aspect('equal', adjustable='box')
plt.title('Алгоритм Коэна-Сазерленда: отсечение отрезков прямоугольным окном')
plt.legend()
plt.grid(True, linestyle=':', alpha=0.6)
plt.show()
```